



Case Studies

📁 Class	GP
📅 Date	@15 November 2025
📎 powerpoint	"Microsoft.pdf" & "Bag System at Denver.pdf"
☑️ revisão?	☐

Os estudos de caso fornecidos, relativos ao **Sistema de Bagagens Automatizado no Aeroporto de Denver** e ao desenvolvimento da aplicação **Word for Windows (Projeto Opus)** na **Microsoft**, oferecem lições cruciais sobre a gestão de projetos complexos, a dificuldade de implementar novas tecnologias em grande escala e a importância dos processos de desenvolvimento.

A. O Sistema de Bagagens no Aeroporto de Denver (DIA)

Este caso sublinha os riscos inerentes à implementação de sistemas complexos e inovadores, especialmente quando operam perto da capacidade máxima.

Complexidade e Risco Tecnológico

- **Risco Inerente a Sistemas Novos e Complexos:** Sistemas automatizados de bagagem são inerentemente arriscados e podem não funcionar corretamente. O sistema de Denver foi originalmente concebido como um "**Integrated Automated Baggage Handling System**" (**IABHS**), único na sua complexidade, tecnologia inovadora e capacidade antecipada.
- **Salto Tecnológico Exponencial:** O sistema representou um enorme **salto tecnológico** sobre as práticas atuais,

comparável a tentar saltar da terceira para a quinta ou sexta geração de sistemas de bagagens. Esta complexidade foi estimada como **100 vezes maior** do que a de sistemas comparáveis, aumentando exponencialmente as dificuldades para o fazer funcionar.

- **Problemas Mecânicos e Profundos:** O sistema foi atormentado por problemas mecânicos visíveis, como os carrinhos emperrarem, desalinhamento com as correias transportadoras e danos nas malas. No entanto, o problema mais fundamental e duradouro era o da "confiabilidade dos tempos de entrega", causado pela sua extrema complexidade e variabilidade de cargas.

Falhas de Design e Planeamento

- **Design Tardiamente Integrado:** O design do sistema de bagagens foi largamente uma **ideia posterior** (*afterthought*), detalhado muito depois de a construção do aeroporto estar em curso e apenas cerca de dois anos antes da abertura prevista.
- **Restrições e Falta de Testes:** O prazo apertado e as restrições de geometria impediram simulações extensivas ou testes físicos do design completo. A implementação de um avanço tão complexo deveria ter sido encarada como um **grande projeto de investigação e desenvolvimento (I&D)**, onde o sucesso não pode ser garantido em pouco tempo.
- **Ausência de Backup (Originalmente):** O design original do sistema totalmente automatizado **não incluía um sistema de backup significativo**.
- **O Problema da "Line Balancing":** O sistema complexo consistia em mais de uma centena de filas de espera interligadas ("cascade of queues"). Um problema crucial era o **"line balancing"** (equilíbrio de linhas) – garantir que todas as linhas de fluxo recebessem um serviço igualmente bom (por exemplo, fornecendo carrinhos vazios suficientes a todas as correias transportadoras). A dificuldade em

resolver este problema aumenta exponencialmente com o número de linhas.

Consequências e Soluções (Curto Prazo)

- **Atrasos e Custos Massivos:** Os problemas do sistema de bagagens causaram um atraso de cerca de 16 meses na abertura do Novo Aeroporto de Denver. Os custos de atraso (juros da dívida, manutenção e custos de oportunidade para as companhias aéreas) foram estimados em cerca de **US\$ 500 milhões** até março de 1995.
- **Redução Drástica da Complexidade:** Para que o sistema funcionasse, a United Airlines (a companhia aérea dominante) teve de **reduzir drasticamente a sua complexidade e desempenho**. O conceito de sistema "totalmente automatizado" para todo o aeroporto foi abandonado.
- **Implementação de Backup:** Foi instalado um sistema alternativo (laborioso e caro de operar) com base em **"tecnologia de correia transportadora padrão com entrega de malas aos concourses por rebocadores e carrinhos"**, a um custo de US\$ 50 a 75 milhões.
- **Redução de Carga:** O sistema automatizado foi reconfigurado para servir apenas 1 dos 3 *concourses*, operar a metade da capacidade planeada em cada trilho (30 sacos/minuto em vez de 60) e, inicialmente, lidar apenas com bagagem de partida (não transferências). Esta redução de carga aumenta a fiabilidade, tal como previsto pela teoria, mas **reduz drasticamente a relação custo-eficácia** do sistema automatizado.

B. Microsoft Corporation: Unidade de Negócios Office (Projeto Opus/WinWord)

Este caso ilustra os desafios no desenvolvimento de software de ponta, as consequências da pressão sobre os cronogramas e a

evolução das metodologias de gestão de projetos.

Atrasos no Desenvolvimento e Pressão de Agendamento

- **Atraso Extremo:** O Projeto Opus (Word for Windows) levou **mais de cinco anos** a ser concluído, sendo lançado em 30 de novembro de 1989, com um atraso significativo em relação à data de envio projetada (setembro de 1985).
- **Fenómeno "Infinite Defects" (Defeitos Infinitos):** A pressão constante sobre os desenvolvedores levava-os a fazer o mínimo de trabalho necessário para demonstrar uma funcionalidade, marcando-a como concluída. Os *bugs* inevitáveis descobertos meses depois eram vistos como não relacionados, e demoravam mais a corrigir porque os desenvolvedores já não se lembravam do código, atrasando ainda mais o cronograma.
- **Sucesso Apesar dos Problemas:** Apesar do longo atraso, o Word for Windows foi aclamado pela crítica, sendo classificado como superior ao concorrente WordPerfect, e as vendas excederam as projeções.

Evolução da Gestão de Projetos e Organização

- **Desenvolvimento Inicial Informal:** Nos primeiros dias, a Microsoft dependia de desenvolvedores "superstars" (estrelas) que muitas vezes trabalhavam sem uma especificação ou design formal.
- **Institucionalização da Gestão de Programas:** A função de **Program Management** (Gestão de Programas) foi introduzida para formalizar o design, coordenar funções (desenvolvimento, testes, educação do utilizador) e atuar como integrador do esforço.
- **Liderança Partilhada:** Em 1990, a liderança de um novo produto era partilhada entre várias pessoas (líder de projeto, líder técnico, gestor de programa, gestor de

marketing, etc.), sem que uma única pessoa tivesse autoridade total.

- **Conflito de Funções:** Havia um debate sobre o papel dos gestores de programa. Alguns sentiam que deveriam evoluir para "designers de software", mas a maioria dos desenvolvedores opunha-se, argumentando que os gestores de programa careciam da sofisticação técnica necessária.

Propostas de Melhoria do Processo

- **Abordagem Estruturada (McDermott):** Defendia duas fases claras: desenvolvimento do conceito (levando a uma especificação completa) e implementação. O objetivo era "**congelar a especificação**" antes de o trabalho de desenvolvimento começar.
- **Abordagem "Zero Defects" (Mason):** Propôs ter um produto funcional e pronto para envio todos os dias. Isso implicava o desenvolvimento em **módulos orientados a objetos** em sequências controladas, permitindo que o produto evoluísse gradualmente e proporcionando flexibilidade para cumprir quase qualquer data de envio (com um *tradeoff* claro em termos de recursos e funcionalidade).

Desafios Estratégicos (Partilha de Código)

- **Partilha de Código (*Code Sharing*):** Havia um esforço para partilhar rotinas comuns (menus, macros, gráficos) entre aplicações para diferentes sistemas operativos (MS-DOS, Windows, Macintosh) para obter eficiências de desenvolvimento e uma interface de utilizador consistente.
- **Os Riscos da Partilha de Código:** A partilha de código no Opus com o MacWord gerou grandes atrasos (talvez seis meses no WinWord e oito meses no MacWord) devido a pequenas alterações de otimização e problemas de compatibilidade/correção de *bugs*. A lição era que **partilhar 20% do código é um pesadelo**; para partilhar com sucesso, é necessário ter um único conjunto de código fonte.

- **Abordagem "Core Code"**: Esta abordagem visava escrever quase toda a funcionalidade de uma aplicação em **código "genérico"** a ser partilhado por todas as plataformas. O risco desta estratégia é que o software tende a ser escrito para o **"denominador comum mais baixo"**, dificultando o uso total das capacidades de cada máquina e podendo resultar num produto que não é o melhor em nenhuma das plataformas.

Conclusão:

O cerne de ambos os estudos de caso – o aeroporto de Denver e o desenvolvimento do software da Microsoft – reside na **gestão da complexidade**.

Se a inovação é como construir um edifício muito alto, o caso de Denver mostra que **tentar saltar gerações tecnológicas sem testar a fundação leva ao colapso do cronograma e a custos enormes**, forçando o projeto a regredir a uma solução mais simples e apoiada por um plano de contingência (o backup manual). O caso da Microsoft, embora em software, mostra que **a complexidade de um novo sistema, quando combinada com a pressão e a falta de uma metodologia estruturada, leva a um turbilhão de defeitos e a atrasos prolongados**, que só podem ser resolvidos através da simplificação radical (como reduzir funcionalidades) ou por um investimento significativo de tempo para refinar e estruturar o processo (como as propostas de *Zero Defects* ou *Core Code*).